SYSTEM REQUIREMENTS SPECIFICATION FOR KCA CONNECT AGENTIC AI

IGNATIUS LUMOSI

22/05647

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY (BIT 4104)

SUPERVISOR: GRIFFIN KENGA

DATE SUBMITTED: 24/10/2025

Table of Contents

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for KCA Connect Agentic AI, an intelligent digital assistant designed to enhance communication and service delivery at KCA University. The purpose of this document is to provide a clear and testable description of the system's expected behaviour, serving as a foundation for design, development, testing, and validation. It ensures alignment between stakeholder needs, technical implementation, and institutional goals.

1.2. Scope

KCA Connect Agentic AI will serve as a centralized, AI-powered communication platform for students, lecturers, and administrators at KCA University. The system will provide real-time access to academic and administrative information—including timetables, fee structures, unit registration details, exam schedules, and institutional announcements—based on the user's selected campus (Main/Town/Kitengela).

The system will feature a conversational interface (chatbot), integrate with existing university data sources, and support personalized, instant responses. It will not include predictive analytics, third-party payment integrations, or external social media linking in its initial release. However, the architecture will be modular to allow future scalability.

1.3. Acronyms and Definitions

Acronym	**Definition**
AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface

2. Overall Description

2.1. System Perspective

KCA Connect Agentic AI will operate as a web-based application integrated into KCA University's existing digital ecosystem. It will function as a standalone service but will consume data from internal university systems (e.g., student portal, academic calendars, fee databases) via secure APIs. The system will not replace existing platforms but will act as a unified, intelligent front-end that simplifies access to fragmented information sources. The architecture includes:

- Frontend: Web interface (HTML, Tailwind CSS, JavaScript)

- AI Engine: Python-based backend using LangChain and Llama LLM

- Data Layer: Firebase (for authentication) and Qdrant (for vector storage of university documents)

2.2. System Environment

- Hardware: Standard university servers or cloud hosting (e.g., Firebase Hosting)

- Software:

  - Backend: Python 3.8+, LangChain, Llama (local or cloud-hosted LLM)

  - Frontend: HTML5, Tailwind CSS, JavaScript

  - Database: Firebase (user auth), Qdrant (semantic search embeddings)

- Network: Requires stable internet connection (minimum 10 Mbps); accessible via university Wi-Fi or mobile data

2.3. User Characteristics

User Summary

1. Students

- Technical Proficiency: Basic to intermediate
- Needs: Quick access to timetables, fees, exam information, and unit registration help

2. Lecturers

- Technical Proficiency: Intermediate
- Needs: Share announcements, quick access to timetable and resources

3. Administrators

- Technical Proficiency: Intermediate to advanced
- Needs: Broadcast messages, manage FAQs, and monitor system usage

General User Characteristics

- All users are comfortable with smartphones and web browsers.
- Users may not understand AI concepts or technical jargon.
- The interface must be intuitive, user-friendly, and require no training.

2.4. Constraints

- Must comply with KCA University's data privacy and IT policies

- Cannot store sensitive personal data (e.g., ID numbers, financial records)

- Development limited to open-source or freely available AI models (e.g., Llama) due to budget

- Must be deployable within 6 months using student resources

- Must support all campus branches contexts

3. Functional Requirements

3.1. User Authentication & Authorization

Input: User enters institutional email and password.

Process:

- System validates credentials against Firebase Authentication
- On successful authentication, system retrieves user role (student/lecturer/admin) and associated metadata (student ID, program, year of study)
- System generates a session token with 24-hour validity
- Failed attempts are logged; after 5 consecutive failures, account is temporarily locked for 15 minutes

Output:

- Valid credentials → User redirected to role-specific dashboard with personalized greeting
- Invalid credentials → Clear error message ("Incorrect email or password. X attempts remaining")
- Locked account → "Too many failed attempts. Try again in 15 minutes" with password reset option

3.2. Campus Selection & Preference Management

Input: User selects campus (Main/Town/Kitengela) during onboarding or via settings menu.

Process:

- System stores campus preference in user profile (Firebase)
- Preference is used as a filter parameter for all subsequent queries
- System allows users to temporarily switch campus context without changing default preference
- Campus selection triggers loading of campus-specific resources (maps, contact info, facility hours)

Output:

- Dashboard displays campus-specific banner/visual identifier
- All information (timetables, announcements, facilities) filtered by selected campus
- Option to view "All Campuses" for cross-campus events or announcements

3.3. Intelligent Query Processing

Input: User submits natural language query via text or voice input (e.g., "When are exams?" or "What's the library closing time today?")

Process:

- NLP engine (LangChain + Llama) parses query to extract intent and entities (dates, course codes, locations)
- System applies context filters: user role, campus, current academic term
- Query is converted to vector embedding and matched against Qdrant vector database
- If confidence score < 70%, system requests clarification or offers similar queries
- System logs all queries for continuous improvement and FAQ generation

Output:

- High confidence (>90%): Direct answer with source citation
- Medium confidence (70-90%): Answer with "Did you mean...?" suggestions
- Low confidence (<70%): "I'm not sure I understood. Did you mean: [suggestions]" or escalation to human support
- Response includes related questions/follow-ups

3.4. Personalized Timetable Access

Input: User requests timetable via query ("Show my timetable", "What classes do I have tomorrow?", "When is my next lecture?")

Process:

- System identifies user from session token and retrieves enrolled units
- Fetches timetable data from academic database API filtered by: user ID, current semester, campus
- Parses schedule data and formats by day/week/month view
- System checks for timetable conflicts, changes, or cancellations since last access
- Highlights upcoming classes (next 2 hours) and allows export options

Output:

- Interactive weekly calendar view with color-coded units
- Details per class: Unit code, name, lecturer, room, time
- Indicators for online vs. physical classes
- Notifications for last-minute changes or cancellations
- Export options: iCal, Google Calendar, PDF

3.5. Comprehensive Fee Inquiry

Input: User queries fees ("How much are fees for BIT?", "What's my fee balance?", "When is the payment deadline?")

Process:

- System determines query type: general fee structure vs. personal fee status
- For general queries: Searches Qdrant for program-specific fee documents
- For personal queries: Fetches user's fee statement from finance API (with proper authorization)
- Cross-references with payment deadlines and available payment plans
- System identifies applicable scholarships, bursaries, or payment options

Output:

- General query: Fee breakdown table (tuition, facilities, activity fees, etc.) by year/semester
- Personal query: Current balance, paid amount, deadline, payment history
- Links to payment portal and financial aid office contact
- Downloadable fee statement (PDF)

3.6. Targeted Announcement & Notification System

Input: Administrator creates announcement via admin panel with targeting parameters (role, campus, program, year)

Process:

- System validates admin credentials and permissions
- Announcement is categorized (academic/administrative/emergency/social) and prioritized
- Targeting rules applied: filter recipients by campus, role, program, or year of study
- Notification dispatched via multiple channels: in-app alert, email (optional), push notification (if mobile app exists)
- System tracks read/unread status and engagement metrics
- Emergency announcements bypass normal filters and reach all users immediately

Output:

- Users receive real-time pop-up notification (dismissible for non-emergency)
- Announcement appears in notification center with timestamp, category badge, and unread indicator
- Full announcement accessible via click with option to mark as read, bookmark, or share

3.7. Comprehensive Exam Schedule & Support

Input: User queries exam information ("When is BIT 0401 exam?", "Show my exam timetable", "Where is my exam venue?",)

Process:

- System parses query to extract course code, exam type (special/final), or general schedule request
- Matches query against exam timetable database filtered by user's enrolled units and campus
- Retrieves exam rules, regulations, and allowed materials from knowledge base
- Checks for exam clashes and notifies user of conflicts
- Provides campus map integration for venue location

Output:

- Specific query: Date, time, venue, duration, and exam type for requested unit
- General query: Complete exam timetable in chronological order with countdown timers
- Venue details: Building name, room number
- Exam regulations: Allowed materials, reporting time, ID requirements
- Clash detection: Alert if multiple exams scheduled at conflicting times

Additional Features:

- Exam preparation resources: past papers, study tips, consultation schedules
- Venue capacity and accessibility information
- Special accommodation requests guidance (for students with disabilities)

4. Non-Functional Requirements

4.1. Performance

Response Time:

- Chat queries shall return responses within **5 seconds** for 95% of requests under normal load (up to 100 concurrent users)
- Complex queries requiring multi-source data aggregation shall complete within **8 seconds** maximum
- Page load time shall not exceed **5.5 seconds** for initial dashboard rendering
- Timetable and schedule displays shall render within **5 seconds** of request

Throughput:

- System shall handle at least **500 queries per minute** during peak hours (8 AM - 5 PM, Monday-Friday)
- Database queries shall execute in under **500 milliseconds** for 99% of operations
- Vector search operations in Qdrant shall complete within **300 milliseconds**

Resource Utilization:

- CPU usage shall remain below **70%** under normal load
- Memory consumption shall not exceed **80%** of allocated resources
- Database connection pool shall maintain **minimum 20 active connections** with auto-scaling capability

Performance Testing Requirements:

- Load testing with **100, 250, 500, and 1,000** concurrent users
- Stress testing to identify breaking point (expected >1,500 users)
- Response time monitoring with **99th percentile** metrics tracked

4.2. Security

Data Transmission Security:

- All communications shall use **TLS 1.3** or higher encryption
- **HTTPS mandatory** for all endpoints; HTTP requests automatically redirected to HTTPS
- SSL certificates shall be valid, properly configured, and auto-renewed before expiry
- API calls shall use **encrypted payload** for sensitive data transmission

Authentication & Authorization:

- Multi-factor authentication (MFA) **optional** for users, **mandatory** for administrators
- Session tokens shall expire after **24 hours** of inactivity
- Refresh tokens valid for **7 days** with automatic rotation
- Password requirements: minimum **8 characters**, including uppercase, lowercase, numbers, and special characters
- **bcrypt** or **Argon2** hashing for any locally stored authentication data
- Role-based access control (RBAC) with principle of least privilege

Data Protection:

- **No plain-text storage** of passwords or authentication credentials
- Personally Identifiable Information (PII) shall be **encrypted at rest** using AES-256
- User data retention policy: inactive accounts archived after **2 years**, deleted after **5 years**
- Audit logging for all data access, modifications, and deletion operations
- Compliance with **GDPR** and Kenya's **Data Protection Act 2019**

API Security:

- API keys rotated **every 90 days**
- Rate limiting: **100 requests per minute per user**, **1,000 per minute per IP**
- Input validation and sanitization to prevent **SQL injection, XSS, and CSRF** attacks
- **CORS** policies strictly configured for authorized domains only

Vulnerability Management:

- Monthly security audits and dependency vulnerability scans
- Automated security patching for critical vulnerabilities within **48 hours**
- Penetration testing conducted **annually** by external security firm
- Security incident response plan with **24-hour escalation** protocol

4.3. Usability

User Interface Design:

- **Clean, minimalist design** following Material Design or similar modern UI principles
- Maximum **3 clicks** to reach any core feature from dashboard
- **Consistent navigation** structure across all pages
- Visual hierarchy with clear headings, labels, and call-to-action buttons
- **Error messages** shall be clear, actionable, and written in plain language (avoid technical jargon)

Learnability:

- First-time users shall complete basic tasks (login, query submission, timetable view) within **5 minutes** without external help
- Interactive **onboarding tutorial** (skippable) for new users highlighting key features
- Contextual tooltips and help icons throughout the interface

Responsiveness:

- Fully responsive design supporting screen sizes from **320px (mobile) to 2560px (4K displays)**
- Touch-friendly interface with **minimum 44x44px touch targets** on mobile
- **Progressive Web App (PWA)** capabilities for offline access and home screen installation
- Adaptive layouts: single-column on mobile, multi-column on desktop

4.4. Reliability

Uptime Requirements:

- System availability: **99.5% uptime** per academic term (approximately 3.6 hours downtime per month)
- Critical services (authentication, query processing): **99.9% uptime** (43 minutes downtime per month)
- Planned maintenance windows: **maximum 4 hours monthly**, scheduled during low-traffic periods (2 AM - 6 AM)

Fault Tolerance:

- **Automatic failover** to backup servers within 30 seconds of primary server failure
- Database replication with **real-time synchronization** (lag <10 seconds)
- Graceful degradation: if AI engine fails, system displays cached responses or fallback to FAQ database
- **Circuit breaker pattern** implemented for external API calls to prevent cascading failures

Error Handling:

- **Zero data loss** guarantee for user-submitted queries and profile changes
- Transaction rollback mechanisms for failed operations
- User-friendly error messages with **unique error codes** for support tracking
- Automatic retry logic (max 3 attempts) for transient failures

Backup & Recovery:

- **Daily automated backups** of all databases at 3 AM
- Backup retention: **daily backups for 7 days, weekly backups for 4 weeks, monthly backups for 1 year**
- **Recovery Point Objective (RPO):** Maximum 24 hours of data loss
- **Recovery Time Objective (RTO):** System restored within 4 hours of catastrophic failure
- Quarterly disaster recovery drills to validate restoration procedures

Monitoring & Alerting:

- **Real-time monitoring** of server health, response times, error rates, and user activity
- Automated alerts for: downtime, performance degradation >20%, error rate >5%, SSL certificate expiration
- **24/7 monitoring** with escalation to technical team within 15 minutes of critical alerts
- Status page accessible to users showing real-time system health

4.5. Scalability

Horizontal Scalability:

- Architecture shall support **auto-scaling** based on demand (CPU >70% or concurrent users >80)
- Load balancing across multiple server instances with **round-robin or least-connections** algorithm
- Stateless application design enabling seamless addition of server nodes
- Target capacity: **1,000 concurrent users** without performance degradation

Vertical Scalability:

- System shall accommodate **10x data growth** (users, queries, documents) within 3 years
- Database shall efficiently handle **100,000+ stored conversations** and **10,000+ documents** in vector database
- Disk storage expandable to **1TB+** without architectural changes

Database Scalability:

- **Qdrant vector database** configured for horizontal scaling with sharding
- Firebase **Firestore** native scalability leveraged for user data
- Read replicas for frequently accessed data to distribute query load
- Connection pooling optimized for high-concurrency scenarios

Future Expansion Readiness:

- Plugin architecture for adding new features without core system modifications
- Support for additional campuses by configuration (no code changes required)
- Integration-ready design for future third-party services (payment gateways, learning management systems)
- Multi-tenancy support for potential expansion beyond KCA University

4.6. Portability

Cross-Browser Compatibility:

- Full functionality on latest versions of:
  - ✓ **Google Chrome** (v110+)
  - ✓ **Mozilla Firefox** (v110+)
  - ✓ **Apple Safari** (v16+)
  - ✓ **Microsoft Edge** (v110+)
- Graceful degradation on older browsers with clear upgrade prompts
- **No browser-specific plugins** required (no Flash, Java applets)

Cross-Platform Support:

- **Desktop:** Windows 10+, macOS 11+, Linux (Ubuntu 20.04+)
- **Mobile:** iOS 14+, Android 10+
- **Tablets:** iPadOS 14+, Android tablets
- Consistent functionality across all platforms (no platform-exclusive features)

Responsive Design:

- **Mobile-first** approach ensuring optimal experience on smartphones
- Breakpoints: **320px, 768px, 1024px, 1440px, 1920px**
- Touch gestures support: swipe, pinch-to-zoom, long-press
- Orientation support: portrait and landscape modes

Device Independence:

- No hardware-specific requirements beyond standard input devices (keyboard, mouse, touchscreen)
- Compatible with **low-end devices:** minimum 2GB RAM, dual-core processor
- Network adaptability: functional on **2G/3G connections** with reduced features (text-only mode)

Technology Portability:

- Backend services containerized using **Docker** for deployment flexibility
- Cloud-agnostic architecture: deployable on **AWS, Google Cloud, Azure,** or on-premises servers
- Database abstraction layer allowing migration between Firebase alternatives if needed
- Standard web technologies (HTML5, CSS3, ES6+) ensuring long-term maintainability

4.7. Availability

Service Level Agreement (SLA):

- **24/7 availability** with guaranteed **99.5% uptime** annually (approximately 43.8 hours downtime per year)
- Peak hours (8 AM - 8 PM, Monday-Friday) target: **99.9% availability**
- Off-peak and weekend hours: **99.0% availability** (allowing for maintenance)

Maintenance Windows:

- Scheduled maintenance: **maximum 4 hours per month**
- Advance notice: **minimum 72 hours** via email, in-app notification, and status page
- Preferred maintenance windows: **Saturdays 2 AM - 6 AM EAT** (East Africa Time)
- Emergency maintenance allowed with **2-hour notice** for critical security patches
- Zero downtime deployments using **blue-green deployment** strategy where possible

Network Resilience:

- Multi-provider network redundancy (primary and backup ISPs)
- **DNS failover** with <60-second TTL for rapid traffic rerouting
- Protection against **DDoS attacks** with traffic scrubbing and rate limiting
- Bandwidth provisioning: minimum **1 Gbps** with burst capacity to **10 Gbps**

High Availability Configuration:

- **Active-passive failover** for critical components
- Health check endpoints monitored every **30 seconds**
- Automatic traffic rerouting within **60 seconds** of failure detection
- No single point of failure in infrastructure design

5. External Requirements

5.1. User Interfaces

5.1.1. Login & Authentication Screen

Components:

a) **Email Input Field**
- Type: Text input with email validation
- Placeholder: "Enter your KCA University email"
- Validation: Real-time validation for @students.kca.ac.ke or @kca.ac.ke domain
- Error states: "Invalid email format" or "Please use your KCA email address"
- Auto-complete support enabled

b) **Password Input Field**
- Type: Password (masked input with toggle visibility icon)
- Placeholder: "Enter your password"
- Minimum length: 12 characters
- Show/Hide password toggle button (eye icon)
- Password strength indicator (weak/medium/strong) during registration

c) **Remember Me Checkbox**
- Optional: Keeps user logged in for 7 days on trusted devices
- Clear explanatory text: "Keep me signed in on this device"

d) **Login Button**
- Primary CTA button with loading spinner during authentication
- Disabled state when form is incomplete or invalid
- Text: "Sign In" or "Signing In..." (during processing)

e) **Forgot Password Link**
- Hyperlink below password field
- Opens password reset modal/page
- Flow: Email input → Verification code sent → New password creation

f) **Sign Up Link**

- For new users: "Don't have an account? Sign Up"
- Links to registration page with student

5.1.2. Main Dashboard

Layout Structure:

- Header Bar (Fixed at top)
- Primary Content Area
    - ✓ Chat Window (Center, 60% width)
    - ✓ Text Input Box (Bottom of chat)
- Sidebar Panel (Right, 30% width, collapsible on mobile)
    - ✓ Quick Actions Card
    - ✓ Upcoming Events Widget
    - ✓ Recent Announcements
- Footer (Optional, bottom of page)

Responsive Behavior:

- **Desktop (>1024px):** Three-column layout (sidebar, chat, quick actions)
- **Tablet (768px-1024px):** Two-column layout (chat + collapsible sidebar)
- **Mobile (<768px):** Single column, hamburger menu for navigation

5.2. Software Interfaces

5.2.1. Firebase Authentication API

Purpose: User authentication, authorization, and session management

Integration Details:

- **SDK:** Firebase JavaScript SDK v10.x
- **Endpoint:** https://identitytoolkit.googleapis.com/v1/
- **Authentication Methods:**
    - ✓ Email/Password authentication (primary)

Key Operations:

- **User Registration**

  - **Request Parameters:**
    - ✓ `email`: String (KCA domain validated)
    - ✓ `password`: String (min 12 chars)
  - **Response:** User object with UID, email, creation timestamp

- **User Login**

  - **Endpoint:** `signInWithEmailAndPassword(auth, email, password)`
  - **Request:** Email + password credentials
  - **Response:**
    - ✓ ID token (JWT, 1-hour validity)
    - ✓ Refresh token (auto-renewal)
    - ✓ User metadata (UID, email, display name)

- **Session Management**
- **Password Reset**
- **User Profile Management**

Security Configuration:

- Email enumeration protection enabled
- Password policy enforcement in Firebase Console
- Custom claims for role-based access (student/lecturer/admin)
- Multi-factor authentication (MFA) optional enablement

Rate Limiting:

- Login attempts: 5 per minute per IP
- Password reset: 3 per hour per email
- Account creation: 10 per hour per IP

Monitoring:

- Firebase Authentication dashboard for user analytics
- Failed login attempt logging
- Unusual activity alerts (new device, location change)

5.2.2. Qdrant Vector Database API

Purpose: Semantic search and retrieval of university documents, FAQs, policies

Integration Details:

- **Version:** Qdrant v1.7+
- **Deployment:** Self-hosted on university servers or Qdrant Cloud
- **Authentication:** API key-based (header: `api-key: <KEY>`)

Collection Structure:

✓ **University Documents Collection**

- **Collection Name:** `kca_documents`
- **Vector Size:** 768 (using sentence-transformers model)
- **Distance Metric:** Cosine similarity

✓ **FAQ Collection**

- **Collection Name:** `kca_faqs`
- **Optimized for:** Quick question-answer retrieval

Key API Operations:

A. Search Query

- **Endpoint:** `POST /collections/{collection_name}/points/search`
- **Response:** Array of matching points with scores and payloads
- **Response Time:** <300ms for 95% of queries

B. Insert/Update Documents

- **Endpoint:** `PUT /collections/{collection_name}/points`
- **Batch Upload:** Up to 100 documents per request
- **Upsert Support:** Automatic document update if ID exists

C. Delete Documents

- **Endpoint:** `POST /collections/{collection_name}/points/delete`
- **Filters:** By document ID or matching criteria

D. Collection Management

- **Create Collection:** `PUT /collections/{collection_name}`
- **List Collections:** `GET /collections`
- **Collection Info:** `GET /collections/{collection_name}`

Embedding Generation:

- **Model:** `sentence-transformers/all-MiniLM-L6-v2` or `paraphrase-multilingual-MiniLM-L12-v2`
- **Process:** User query → Embedding API → Vector → Qdrant search
- **Caching:** Frequently accessed queries cached for 15 minutes

Performance Optimization:

- **Indexing:** HNSW (Hierarchical Navigable Small World) for fast approximate nearest neighbour search
- **Quantization:** Optional scalar quantization to reduce memory usage
- **Sharding:** Distribution across multiple nodes for large datasets

Data Management:

- **Document Ingestion Pipeline:**
  - ✓ Source files (PDF, DOCX) → Text extraction → Chunking (500-word chunks with overlap) → Embedding generation → Qdrant upload
- **Scheduled Updates:** Weekly crawl of KCA website for new announcements/policies
- **Version Control:** Document versioning with timestamp tracking

5.2.3. Academic Data API (Mock/Integration Layer)

Purpose: Retrieve student-specific and institutional data (timetables, fees, exam schedules)

Integration Details:

- **Base URL:** `https://api-mock.kca.ac.ke/v1/` (development) or integration with actual KCA systems
- **Authentication:** OAuth 2.0 or API key with user scope
- **Rate Limiting:** 100 requests/minute per user

API Endpoints:

A. Timetable API

- **Endpoint:** `GET /students/{student_id}/timetable`
- **Query Parameters:**
  - ✓ `campus`: main | town | kitengela
  - ✓ `semester`: current | specific date
  - ✓ `week`: ISO week number (optional)

B. Fee Statement API

- **Endpoint:** `GET /students/{student_id}/fees`
- **Query Parameters:**
  - ✓ `academic_year`: 2024-2025
  - ✓ `semester`: optional filter

C. Exam Schedule API

- **Endpoint:** `GET /students/{student_id}/exams`
- **Alternate:** `GET /exams?unit_code=CS301&campus=main`

D. Unit Registration API

- **Endpoint:** `GET /students/{student_id}/registration-status`
- **Response:** Registered units, pending units, registration deadlines
- **POST Endpoint (Future):** `POST /students/{student_id}/register-units` for direct registration

E. Announcements API

- **Endpoint:** `GET /announcements`
- **Query Parameters:**
  - ✓ `campus`: filter by campus

    - ✓ `category`: academic | administrative | event
    - ✓ `date_from`, `date_to`: date range filter
- **Response:** Array of announcement objects with title, content, publish date, priority

Mock Data Strategy (Development Phase):

- JSON files stored locally simulating API responses
- Faker.js for generating realistic test data
- Configurable delays to simulate network latency
- Error simulation for testing error handling

Production Integration:

- Coordination with KCA IT department for API access
- Data refresh intervals: Timetables (daily), Fees (hourly), Exams (daily)
- Caching strategy to reduce API load

5.2.4. LangChain Framework

Purpose: Orchestration layer connecting LLM (Llama) with data sources and managing conversation flow

Integration Details:

- **Library:** LangChain Python v0.1.x
- **LLM Model:** Llama 3 8B (local deployment) or Llama API
- **Deployment:** Python FastAPI backend

Core Components:

A. Conversation Chain

- **Chain Type:** `ConversationalRetrievalChain`
- **Memory:** `ConversationBufferMemory` (stores last 10 exchanges)

B. Document Retrieval

- **Retriever:** Qdrant vector store retriever
- **Retrieval Strategy:** Maximum Marginal Relevance (MMR) to ensure diverse results
- **Retrieval Parameters:**
  - ✓ `k=5`: Top 5 most relevant documents
  - ✓ `fetch_k=20`: Fetch 20 candidates before MMR selection
  - ✓ `lambda_mult=0.7`: Balance between relevance and diversity

C. Query Processing Pipeline

1. **Input:** User's natural language query
2. **Query Transformation:** Rephrase for better retrieval (using LLM)
3. **Intent Classification:** Categorize query (timetable, fees, exams, general)
4. **Context Retrieval:**
   - ✓ Vector search in Qdrant
   - ✓ API call to Academic Data API (if student-specific)
5. **Response Generation:** LLM synthesizes response from context
6. **Post-Processing:** Format response, add citations, extract action items
7. **Output:** Structured response with metadata

D. Agent Tools (Advanced)

- **Calculator Tool:** For fee calculations, date arithmetic
- **Web Search Tool:** Fallback for questions outside knowledge base (future)
- **SQL Tool:** Query structured databases (if needed)

Performance Optimization:

- **Streaming Responses:** Token-by-token streaming for faster perceived response time
- **Caching:** Cache responses for identical queries (15-minute TTL)
- **Batch Processing:** Queue multiple queries for efficient processing

Quality Assurance:

- **Response Validation:** Check for hallucinations, irrelevant responses
- **Source Attribution:** Always cite which documents informed the response
- **Fallback Handling:** If confidence < 70%, ask clarifying questions or suggest FAQ

5.2.5. Additional Software Interfaces

A. Email Service (SendGrid or AWS SES)

- **Purpose:** Password resets, notifications, announcements
- **API:** RESTful HTTP API
- **Rate Limit:** 100 emails/hour (configurable)

B. SMS Service (Africa's Talking or Twilio)

- **Purpose:** Critical alerts, 2FA codes
- **Endpoints:** Send SMS, delivery status webhook
- **Cost Consideration:** Use sparingly due to per-message charges

C. Analytics Service (Google Analytics 4 or Mixpanel)

- **Purpose:** User behavior tracking, feature usage analytics
- **Integration:** JavaScript SDK on frontend
- **Events Tracked:** Page views, query submissions, button clicks, errors

D. Logging & Monitoring (ELK Stack or Datadog)

- **Purpose:** Error tracking, performance monitoring
- **Components:** Elasticsearch (storage), Logstash (processing), Kibana (visualization)
- **Log Levels:** DEBUG, INFO, WARNING, ERROR, CRITICAL

E. File Storage (Firebase Storage or AWS S3)

- **Purpose:** User profile pictures, uploaded documents (future)
- **Access Control:** Signed URLs for secure access
- **Quota:** 10MB per user, 1GB total (initial)

5.3. Hardware Interfaces

5.3.1. Server Infrastructure

Development Environment:

- **Local Development:** Developer laptops (Windows/Mac/Linux)
  - ✓ Minimum specs: 8GB RAM, quad-core CPU, 50GB free storage
  - ✓ Docker containers for consistent environment

Staging Environment:

- **Cloud VM:** Single instance for testing
  - ✓ Provider: Google Cloud Platform, AWS, or DigitalOcean
  - ✓ Specs: 4 vCPUs, 16GB RAM, 100GB SSD
  - ✓ OS: Ubuntu 22.04 LTS

Production Environment:

- **Application Servers (2+ instances for redundancy):**
  - ✓ Specs: 8 vCPUs, 32GB RAM, 200GB SSD each
  - ✓ Load balancer distributing traffic (NGINX or cloud-native)
  - ✓ Auto-scaling: Add instance when CPU >75% for 5 minutes
- **Database Servers:**
  - ✓ **Firebase:** Fully managed cloud service (no hardware management)
  - ✓ **Qdrant:** Dedicated instance with 16GB RAM, 4 vCPUs, 500GB SSD for vector storage
- **LLM Inference Server (if self-hosted):**
  - ✓ **GPU Recommendation:** NVIDIA A100 (40GB) or A10G (24GB) for optimal Llama 3 8B performance
  - ✓ **Alternative:** CPU-only deployment on high-core-count server (16+ cores, 64GB RAM)
  - ✓ **Quantization:** Use 4-bit or 8-bit quantized models to reduce memory requirements
- **Backup Storage:**
  - ✓ Network-attached storage (NAS) or cloud object storage (S3/Cloud Storage)
  - ✓ Minimum 1TB capacity with automatic replication

Network Infrastructure:

- **Bandwidth:** Minimum 1 Gbps connection with burst to 10 Gbps
- **CDN:** Cloudflare or AWS CloudFront for static asset delivery
- **DNS:** Cloudflare DNS with DDoS protection
- **SSL/TLS:** Let's Encrypt certificates with auto-renewal

5.3.2. User Devices

Supported Hardware:

- **Desktop/Laptop Computers:**
    - ✓ Minimum: Dual-core CPU, 4GB RAM, 720p display
    - ✓ Recommended: Quad-core CPU, 8GB RAM, 1080p display
    - ✓ Storage: No local storage required (web-based)
    - ✓ Peripherals: Keyboard, mouse/trackpad (standard USB/Bluetooth)
- **Smartphones:**
    - ✓ iOS: iPhone 8 or newer (iOS 14+)
    - ✓ Android: Devices from 2018+ with Android 10+
    - ✓ Screen size: 4.7" minimum for usable experience
    - ✓ No minimum storage (web app, no installation)
- **Tablets:**
    - ✓ iPad (5th gen or newer), iPad Air, iPad Pro
    - ✓ Android tablets (Samsung Galaxy Tab, Lenovo, etc.)
    - ✓ Screen size: 7" minimum recommended

Network Requirements:

- **Minimum Connection:** 2G/3G mobile data or 1 Mbps Wi-Fi
- **Recommended:** 4G LTE or 10 Mbps+ Wi-Fi for optimal experience
- **Offline Capability (Future):** Progressive Web App (PWA) features for limited offline access

6. References

- Deakin University. (2018). Genie digital assistant: Transforming student experience with AI. https://www.deakin.edu.au

- Holmes, W., Bialik, M., & Fadel, C. (2019). Artificial Intelligence in Education: Promises and Implications for Teaching and Learning. Centre for Curriculum Redesign.

- Page, L. C., & Gehlbach, H. (2017). How Georgia State University used an AI chatbot to help students navigate enrolment. Education Next, 17(4), 68–74.

- KCA University IT Policy Guidelines (2025, internal document)

- LangChain Documentation. https://www.langchain.com

- Qdrant Vector Search Engine. https://qdrant.tech